Web Development – Exam 1 (WEB25-19-25) – with answers

The exam is composed of short answer (max. 50 words), code comprehension, and multiple choice questions. Duration: two hours. Max 50 points.
3-5 points for each code comprehension question, 1 point for each multiple-choice question.

Grade	Points
U	< 20
3	20-29
4	30-39
5	40-50

How to read this file: the green ● option is the correct answer. The grey text in parentheses is the Swedish translation.
(Så läser du filen: det gröna ●-alternativet är rätt svar. Den grå texten inom parentes är svensk översättning.)

---

1 T1_General_Q1

Which of the following best describes a web application?
(Vilket av följande beskriver bäst en webbapplikation?)

Select one alternative:

- ○ A native app installed on a mobile device
- ● **Software that runs entirely on a browser, communicating with a server ✔**
- ○ A command-line utility
- ○ A standalone desktop program

Answer:
A web app runs in the browser and talks to a server. You don't install it like a normal app.
(En webbapp körs i webbläsaren och pratar med en server. Du installerar den inte som en vanlig app.)

Totalpoäng: 1

---

2 T1_General_Q2

Which of the following correctly describes HTTP's stateless nature?
(Vilket av följande beskriver korrekt att HTTP är 'stateless' (tillståndslöst)?)

Select one alternative:

- ○ Sessions automatically persist without cookies
- ○ Clients must maintain persistent connections
- ● **Each request is independent and contains all necessary information ✔**
- ○ The server stores all session data

Answer:
The server does not remember earlier requests. Every request must carry everything it needs. That is why we use cookies or tokens to remember a login.
(Servern kommer inte ihåg tidigare requests. Varje request måste ha med allt som behövs. Därför använder man cookies eller tokens för att komma ihåg en inloggning.)

Totalpoäng: 1

3 T1_General_Q3

Given this pseudo-code, what will be printed? Please also provide a short explanation.
(Vad skrivs ut av den här koden? Ge också en kort förklaring.)

```
console.log("Start");
setTimeout(() => console.log("Async Event"), 0);
console.log("End");
```

Answer:
Output: Start, End, Async Event.
setTimeout puts the function in a queue. It only runs when all normal code is finished, even if the time is 0 ms. So "End" is printed before "Async Event".

(Utskrift: Start, End, Async Event. setTimeout lägger funktionen i en kö. Den körs först när all vanlig kod är klar, även om tiden är 0 ms. Därför skrivs "End" ut före "Async Event".)

Totalpoäng: 1

4 T2_HTML/CSS/JS_Q1

Which HTML tag is used to create a hyperlink?
(Vilken HTML-tagg används för att skapa en länk?)

Answer:
<a> (example: Click). "a" means anchor. href tells where the link goes. (The options were not visible in the PDF.)

(<a>. "a" står för anchor (ankare). href säger vart länken går. (Alternativen syntes inte i PDF:en.))

Totalpoäng: 1

5 T2_HTML/CSS/JS_Q2

Which of the following CSS selectors has the highest specificity?
(Vilken av följande CSS-selektorer har högst specificitet (väger tyngst)?)

Select one alternative:

- ○ p.content
- ● **#main p ✔**
- ○ .content p
- ○ div > p

Answer:
An ID (#) is stronger than a class (.), and a class is stronger than a tag. #main p is the only one with an ID.

(Ett ID (#) väger mer än en klass (.), och en klass väger mer än en tagg. #main p är den enda med ett ID.)

Totalpoäng: 1

6 T2_HTML/CSS/JS_Q3

What will be logged to the console?
(Vad loggas i konsolen?)

```
let x = 5;
let y = "5";
console.log(x == y);
```

Select one alternative:

- ○ undefined
- ○ Error
- ○ false
- ● **true ✔**

Answer:
== compares only the value and converts the types first ("5" becomes 5). So 5 == 5 is true. With === (also checks type) it would be false.

(== jämför bara värdet och gör om typerna först ("5" blir 5). Alltså 5 == 5 är true. Med === (kollar även typ) hade det blivit false.)

Totalpoäng: 1

7 T2_HTML/CSS/JS_Q4

What is the output of this code?
(Vad blir utskriften från koden?)

```
for (var i = 0; i < 3; i++) {
  setTimeout(() => console.log(i), 0);
}
```

Select one alternative:

- ● **3 3 3 ✔**
- ○ 1 2 3
- ○ undefined undefined undefined
- ○ 0 1 2

Answer:
var gives ONE shared variable i for the whole loop. The timeouts run after the loop is done, and then i is 3. With let you would get 0 1 2, because let makes a new i each loop.

(var ger EN delad variabel i för hela loopen. Timeouts körs efter att loopen är klar, och då är i = 3. Med let hade det blivit 0 1 2, eftersom let skapar ett nytt i varje varv.)

Totalpoäng: 1

8 T2_HTML/CSS/JS_Q5

What is logged to the console?
(Vad loggas i konsolen?)

```
console.log('start');
setTimeout(() => console.log('timeout'), 0);
Promise.resolve().then(() => console.log('promise'));
console.log('end');
```

Select one alternative:

- ○ start → end → timeout → promise
- ○ start → timeout → promise → end
- ○ start → promise → end → timeout
- ● **start → end → promise → timeout ✔**

Answer:
Order: 1) normal code (start, end), 2) Promises (microtasks), 3) setTimeout (macrotasks). Promise always comes before setTimeout.

(Ordning: 1) vanlig kod (start, end), 2) Promises (microtasks), 3) setTimeout (macrotasks). Promise kommer alltid före setTimeout.)

Totalpoäng: 1

9 T3_Front-end Programming/React_Q1

Which of the following correctly defines a functional component?
(Vilket av följande definierar en funktionskomponent korrekt?)

Select one alternative:

- ● **function MyComp() { return <h1>Hello</h1>; } ✔**
- ○ const MyComp = "Hello"
- ○ <MyComp></MyComp>
- ○ class MyComp extends Component {}

Answer:
A functional component is a normal function that returns JSX. "class ... extends Component" is a class component, not a functional one. <MyComp></MyComp> only USES a component.

(En funktionskomponent är en vanlig funktion som returnerar JSX. "class ... extends Component" är en klasskomponent. <MyComp></MyComp> bara ANVÄNDER en komponent.)

Totalpoäng: 1

10 T3_Front-end Programming/React_Q2

Which hook is used for performing side effects such as data fetching?
(Vilken hook används för sidoeffekter som att hämta data?)

Select one alternative:

- ○ useCallback
- ○ useRef
- ● **useEffect ✔**
- ○ useState

Answer:
useEffect runs after the component renders. That is where you do fetch, timers, etc.
(useEffect körs efter att komponenten renderats. Där gör man fetch, timers osv.)

Totalpoäng: 1

11 T3_Front-end Programming/React_Q3

What is the output in the console?
(Vad blir utskriften i konsolen?)

```
function App() {
  const [text, setText] = React.useState("Hi");

  React.useEffect(() => {
    console.log(text);
  }, [text]);

  return <button onClick={() => setText("Bye")}>Click</button>;
}
```

Select one alternative:

- ● **Logs "Hi" then "Bye" after click ✔**
- ○ Logs "Hi" only
- ○ Logs "Bye" only
- ○ Logs "Bye" then "Hi" after click

Answer:
The effect runs after the first render and logs "Hi". The click changes text to "Bye". text is in the dependency array, so the effect runs again and logs "Bye".
(Effekten körs efter första renderingen och loggar "Hi". Klicket ändrar text till "Bye". text finns i dependency-arrayen, så effekten körs igen och loggar "Bye".)

Totalpoäng: 1

12 T3_Front-end Programming/React_Q4

What is the correct way to share state between unrelated components?
(Vad är rätt sätt att dela state mellan komponenter som inte hänger ihop?)

Select one alternative:

- ○ Pass props directly
- ● **Use context ✓**
- ○ Call useEffect in both components
- ○ Create a new component with shared variables

Answer:
Context makes a value available to all components inside a Provider, without passing props through every level.

(Context gör ett värde tillgängligt för alla komponenter inuti en Provider, utan att skicka props genom varje nivå.)

Totalpoäng: 1

13 T3_Front-end Programming/React_Q5

Fix the bug in this component so the button increments count correctly:
(Fixa buggen så att knappen ökar count korrekt:)

```
const App = () => {
  const [count, setCount] = React.useState(0);
  return <button onClick={count++}>{count}</button>;
};
```

Answer:
Two problems: 1) count++ runs directly during render, not on click. onClick needs a function. 2) You must never change state directly. Use setCount so React re-renders. Fix:

```
<button onClick={() => setCount(count + 1)}>{count}</button>
```

(Två problem: 1) count++ körs direkt vid rendering, inte vid klick. onClick behöver en funktion. 2) Man får aldrig ändra state direkt. Använd setCount så React renderar om.)

Totalpoäng: 1

14 T4_BackEnd/Node.js_Q1

What is Node.js?
(Vad är Node.js?)

Select one alternative:

- ○ A version control tool
- ○ A frontend JavaScript framework
- ○ A database system
- ● **A runtime environment for executing JavaScript on the server ✓**

Answer:
Node lets you run JavaScript outside the browser, for example as a server. (Express is the framework, Node is the runtime.)
(Node låter dig köra JavaScript utanför webbläsaren, t.ex. som server. (Express är ramverket, Node är körmiljön.))

Totalpoäng: 1

15 T4_BackEnd/Node.js_Q2

Which of the following statements about Express routing is true?
(Vilket påstående om routing i Express är sant?)

Select one alternative:

- ● **Each route corresponds to an HTTP method and path ✓**
- ○ Routes can only handle GET requests
- ○ Routes can't send JSON responses
- ○ Routes must be declared after middleware

Answer:
app.get('/users', ...) = method GET + path /users. Routes can also handle POST, PUT, DELETE and send JSON.
(app.get('/users', ...) = metoden GET + sökvägen /users. Routes kan också hantera POST, PUT, DELETE och skicka JSON.)

Totalpoäng: 1

16 T4_BackEnd/Node.js_Q3

Which of the following best describes middleware chaining in Express?
(Vad beskriver bäst 'middleware chaining' i Express?)

Select one alternative:

- ○ Only one middleware can exist per route
- ○ Middleware must always be asynchronous
- ● **Multiple middlewares are defined in a sequence and executed one after another ✓**
- ○ Middleware runs in random order

Answer:
Middleware runs in the order it is registered. Each one calls next() to pass on to the next one.
(Middleware körs i den ordning den är registrerad. Var och en anropar next() för att skicka vidare till nästa.)

Totalpoäng: 1

17 T4_BackEnd/Node.js_Q4

What will this code output in the console when /api is requested?
(Vad skriver koden ut i konsolen när /api anropas?)

```
app.use('/api', (req, res, next) => {
  console.log('API middleware');
  next();
});
app.get('/api', (req, res) => {
  console.log('API route');
  res.send('Done');
});
```

Select one alternative:

- ○ Only "API middleware"
- ○ Error: next() not defined
- ○ Only "API route"
- ● **"API middleware" then "API route" ✔**

Answer:
The middleware is registered first, so it runs first. next() passes the request on to the route, which logs and sends the answer.
(Middleware är registrerad först, så den körs först. next() skickar vidare till routen, som loggar och skickar svaret.)

Totalpoäng: 1

18 T4_BackEnd/Node.js_Q5

What will the following code output when / is requested?
(Vad blir resultatet när / anropas?)

```
app.use(async (req, res, next) => {
  await new Promise(r => setTimeout(r, 100));
  next();
});
app.use((req, res) => res.send('Done'));
console.log('Server ready');
```

Select one alternative:

- ● **"Server ready" then delayed "Done" response ✔**
- ○ Error: async not supported
- ○ "Server ready" never logs
- ○ "Done" printed first then "Server ready"

Answer:
console.log runs right away when the file starts. The middleware only runs when a request comes, and then waits 100 ms before "Done" is sent.
(console.log körs direkt när filen startar. Middleware körs först när en request kommer, och väntar då 100 ms innan "Done" skickas.)

Totalpoäng: 1

19 T5_Browser/server_communication_Q1

What is the correct HTTP method to update an existing resource completely in REST?
(Vilken HTTP-metod används för att uppdatera en befintlig resurs helt i REST?)

Select one alternative:

- ○ POST
- ○ PATCH
- ○ UPDATE
- ● **PUT ✔**

Answer:
PUT = replace the whole resource. PATCH = change only a part. POST = create new. UPDATE does not exist.
(PUT = ersätt hela resursen. PATCH = ändra bara en del. POST = skapa ny. UPDATE finns inte.)

Totalpoäng: 1

20 T5_Browser/server_communication_Q2

What will the following Fetch API code do?
(Vad gör den här fetch-koden?)

```
fetch('https://api.example.com/data')
  .then(response => response.json())
  .then(data => console.log(data));
```

Select one alternative:

- ● **Fetches JSON data from the API and logs it ✔**
- ○ Deletes a resource on the server
- ○ Sends data to the server
- ○ Uploads a file

Answer:
fetch does a GET request by default. .json() turns the answer into a JS object, and then it is logged.
(fetch gör en GET-request som standard. .json() gör om svaret till ett JS-objekt, och sen loggas det.)

Totalpoäng: 1

21 T5_Browser/server_communication_Q3

What is the main purpose of CORS (Cross-Origin Resource Sharing)?
(Vad är huvudsyftet med CORS?)

Select one alternative:

- ● **To allow browsers to access APIs hosted on different domains ✓**
- ○ To encrypt HTTP data
- ○ To cache responses locally
- ○ To reduce server load

Answer:
The browser normally blocks requests to other domains. CORS headers from the server say "this is OK".

(Webbläsaren blockerar normalt anrop till andra domäner. CORS-headers från servern säger "det här är okej".)

Totalpoäng: 1

22 T5_Browser/server_communication_Q4

What does this code log if the server returns a 404 response?
(Vad loggar koden om servern svarar med 404?)

```
fetch('https://api.example.com/data')
  .then(res => {
    if (!res.ok) throw new Error('Not Found');
    return res.json();
  })
  .catch(err => console.log(err.message));
```

Select one alternative:

- ● **Not Found ✓**
- ○ The JSON data
- ○ 404
- ○ undefined

Answer:
With 404, res.ok is false. So the code throws its own Error('Not Found'). .catch catches it and logs err.message = "Not Found".

(Vid 404 är res.ok false. Då kastar koden ett eget Error('Not Found'). .catch fångar det och loggar err.message = "Not Found".)

Totalpoäng: 1

23 T6_Database_Q1

What is MongoDB?
(Vad är MongoDB?)

Select one alternative:

- ● **A NoSQL document database ✔**
- ○ A relational database
- ○ A file system
- ○ A key-value store

Answer:
MongoDB stores data as documents (like JSON), not in tables like SQL.
(MongoDB sparar data som dokument (som JSON), inte i tabeller som SQL.)

Totalpoäng: 1

24 T6_Database_Q2

Which of the following is a key advantage of NoSQL databases?
(Vilken är en viktig fördel med NoSQL-databaser?)

Select one alternative:

- ● **Scalability and flexibility ✔**
- ○ Strong ACID transactions by default
- ○ Fixed schema
- ○ Complex joins

Answer:
No fixed schema and easy to spread over many servers. ACID, fixed schema and joins are strengths of SQL.
(Inget fast schema och lätt att sprida ut på många servrar. ACID, fast schema och joins är SQL:s styrkor.)

Totalpoäng: 1

25 T6_Database_Q3

What is schema design in MongoDB primarily concerned with?
(Vad handlar schemadesign i MongoDB främst om?)

Select one alternative:

- ○ Enforcing fixed field types
- ○ Matching SQL schemas
- ● **Optimizing data structure for query patterns ✔**
- ○ Avoiding indexes

Answer:
You shape the documents after how you will read the data (for example, put data inside the same document if you always fetch it together).
(Man formar dokumenten efter hur datan kommer läsas (t.ex. lägga data i samma dokument om man alltid hämtar det tillsammans).)

Totalpoäng: 1

26 T7_Security_Q1

Which web application weakness does this comic refer to? (xkcd 'Bobby Tables': a son named Robert'); DROP TABLE Students;--)

(Vilken svaghet i webbapplikationer syftar serien på? (Sonen heter Robert'); DROP TABLE Students;--))

Select one alternative:

- ● **SQL injection ✔**
- ○ Cross-site scripting (XSS)
- ○ Unprotected Storage of Credentials
- ○ Improper Authentication

Answer:

The name contains SQL code. The school puts the name straight into an SQL query without cleaning it, so the code runs and deletes the table. Fix: sanitize input / use parameterized queries.

(Namnet innehåller SQL-kod. Skolan lägger namnet direkt i en SQL-fråga utan att rensa det, så koden körs och raderar tabellen. Lösning: sanera input / använd parametriserade frågor.)

Totalpoäng: 1

27 T7_Security_Q2

In the STRIDE threat modelling framework, what does the R (Repudiation) stand for?

(I STRIDE-modellen, vad står R (Repudiation) för?)

Select one alternative:

- ○ Threat action attempting to deny access to valid users, such as by making a web server temporarily unavailable or unusable.
- ○ Threat action aimed at accessing and use of another user's credentials, such as username and password.
- ○ Threat action intending to read a file that one was not granted access to, or to read data in transit.
- ● **Threat action aimed at performing prohibited operations in a system that lacks the ability to trace the operations. ✔**

Answer:

Repudiation = someone can deny they did something, because the system does not log or trace actions. The other options are DoS, Spoofing and Information disclosure.

(Repudiation = någon kan förneka att de gjort något, eftersom systemet inte loggar eller spårar. De andra alternativen är DoS, Spoofing och Information disclosure.)

Totalpoäng: 1

28 T8_Performance_Q1

Which of the following best describes a Content Delivery Network (CDN)?
(Vad beskriver bäst en CDN (Content Delivery Network)?)

Select one alternative:

- ● **A global network of servers that deliver content from the nearest location ✔**
- ○ A single high-speed server
- ○ A local file storage system
- ○ A data compression tool

Answer:
Copies of the files are on servers all over the world. The user gets the file from the nearest server, so it is faster.
(Kopior av filerna finns på servrar över hela världen. Användaren får filen från närmaste server, så det går snabbare.)

Totalpoäng: 1

29 New Question

What is lazy loading in web performance optimization?
(Vad är lazy loading inom prestandaoptimering?)

Select one alternative:

- ○ Preloading all assets on startup
- ○ Caching everything before rendering
- ○ Loading all resources at once
- ● **Loading images or scripts only when needed ✔**

Answer:
Load things only when they are needed (for example an image when you scroll to it). The page starts faster.
(Ladda saker först när de behövs (t.ex. en bild när man scrollar dit). Sidan startar snabbare.)

Totalpoäng: 1

30 T9_StateManagement_Q1

Which is true about state updates in React?
(Vad stämmer om state-uppdateringar i React?)

Select one alternative:

- ● **They are asynchronous and batched ✔**
- ○ They require Redux
- ○ They block rendering
- ○ They happen immediately

Answer:
setState does not change the value right away. React collects updates together (batching) and re-renders after.
(setState ändrar inte värdet direkt. React samlar ihop uppdateringar (batching) och renderar om efteråt.)

Totalpoäng: 1

31 T9_StateManagement_Q2

What happens when the `value` of a Context.Provider changes?
(Vad händer när `value` i en Context.Provider ändras?)

Select one alternative:

- ○ Only the Provider re-renders
- ○ The DOM is cleared
- ● **All components using that context re-render ✔**
- ○ Nothing changes until a refresh

Answer:
All components that read the context (with useContext) get the new value and re-render.
(Alla komponenter som läser contexten (med useContext) får det nya värdet och renderas om.)

Totalpoäng: 1

32 T10_CodeComprehension_Q1

Explain why the output is undefined
(Förklara varför utskriften blir undefined)

```
function test() {
  console.log(x);
  var x = 5;
}
test();
```

Answer:
Because of hoisting. The declaration "var x" is moved to the top of the function and gets the value undefined. The assignment "= 5" stays where it is. So when console.log runs, x exists but is still undefined. JS reads it like this:

```
function test() {
  var x;          // hoisted, x = undefined
  console.log(x); // undefined
  x = 5;
}
```

(På grund av hoisting. Deklarationen "var x" flyttas upp till toppen av funktionen och får värdet undefined. Tilldelningen "= 5" stannar där den står. När console.log körs finns x, men är fortfarande undefined.)

Totalpoäng: 4

33 T10_CodeComprehension_Q2

What happens when this component renders? Please also provide a short explanation.
(Vad händer när komponenten renderas? Ge också en kort förklaring.)

```
function App() {
  const [count, setCount] = React.useState(0);
  React.useEffect(() => setCount(count + 1), [count]);
  return <p>{count}</p>;
}
```

Answer:
Infinite loop: the number keeps going up (0, 1, 2, 3 ...) and React warns "Maximum update depth exceeded".
After each render the effect runs and changes count. count changes, so the component re-renders. count is in the dependency array, so the effect runs again, and so on forever. Fix: use [] or don't set the same state inside the effect.

(Oändlig loop: siffran ökar hela tiden (0, 1, 2, 3 ...) och React varnar "Maximum update depth exceeded". Efter varje rendering körs effekten och ändrar count. count ändras, så komponenten renderas om. count finns i dependency-arrayen, så effekten körs igen, och så vidare för alltid. Lösning: använd [] eller sätt inte samma state i effekten.)

Totalpoäng: 5

34 T10_CodeComprehension_Q3

What does this render? Please also provide a short explanation.
(Vad renderas? Ge också en kort förklaring.)

```
function App() {
  const show = true;
  return show && <h1>Visible</h1>;
}
```

Answer:
It renders <h1>Visible</h1> (a heading with the text "Visible").
&& gives back the second value if the first one is true. show is true, so the JSX is returned. If show was false, it would return false, and React draws nothing for false.

(Den renderar <h1>Visible</h1> (en rubrik med texten "Visible"). && ger tillbaka det andra värdet om det första är sant. show är true, så JSX:en returneras. Om show hade varit false skulle false returneras, och React ritar ingenting för false.)

Totalpoäng: 5

35 T10_CodeComprehension_Q4

Explain why the text disappears after 2 seconds

(Förklara varför texten försvinner efter 2 sekunder)

```
function Message() {
  const [visible, setVisible] = React.useState(true);
  React.useEffect(() => {
    const id = setTimeout(() => setVisible(false), 2000);
    return () => clearTimeout(id);
  }, []);
  return visible ? <p>Hello!</p> : null;
}
```

Answer:

useEffect with [] runs once after the first render and starts a 2000 ms timer. When the timer fires, setVisible(false) is called. The component re-renders, visible is now false, so it returns null and nothing is shown. The cleanup (clearTimeout) stops the timer if the component is removed before 2 seconds.

(useEffect med [] körs en gång efter första renderingen och startar en timer på 2000 ms. När timern går anropas setVisible(false). Komponenten renderas om, visible är nu false, så den returnerar null och inget visas. Cleanup (clearTimeout) stoppar timern om komponenten tas bort innan 2 sekunder.)

Totalpoäng: 5